

ClientFlow AI

PHASE 2: Core Freelance Workflow

Complete Implementation Specification

Frontend | Backend | Database | Workflows

Phase:	2 of 5
Timeline:	Week 3-5 (3 weeks)
Build On:	Phase 1 (Auth + Basic Jobs)
Delivers:	AI Proposals + Marketing + Payments
Status:	Ready for Development

Phase 2 Contents

Feature	Description	Pages
2.1 AI Proposal Generator	OpenAI integration, prompt engineering, UI	3-15
2.2 Status Tracking System	Update jobs, auto follow-ups, reminders	16-22
2.3 Proposal Storage	Save proposals, version history	23-27
2.4 Landing Page	Marketing site, conversion funnel	28-38
2.5 Pricing & Payments	Razorpay integration, subscriptions	39-50
2.6 Upgrade Flow	Free to Pro conversion	51-56
Complete Workflows	End-to-end user journeys	57-60

Feature 2.1: AI Proposal Generator

Timeline: Days 1-7 | The core value proposition of ClientFlow

OVERVIEW

Purpose: Generate personalized, high-quality proposals using GPT-4

User Value:

- Save 15-30 minutes per proposal
- Consistent quality and tone
- Learn from past successes
- Optimize for higher response rates

Technical Approach:

- Use GPT-4 for generation (not GPT-3.5 - quality matters)
- System prompt engineering for consistency
- User context injection (skills, experience, past wins)
- Configurable parameters (tone, length, highlights)
- Free tier: 5 generations/month
- Pro tier: Unlimited

FRONTEND SPECIFICATION

Page 1: Proposal Generator Main Page

Route: /app/proposals/generate/[job_id]

Page Layout: Two-column split screen

LEFT COLUMN (40% width):

- Fixed position (stays visible while scrolling)
- Job context panel
- AI settings panel

RIGHT COLUMN (60% width):

- Scrollable
- Generated proposal display
- Action buttons

=== LEFT COLUMN DETAILS ===

Job Context Panel:

1. Header Section

- Job title (bold, large, 18px)
- Platform badge (Upwork/Fiverr/LinkedIn with logo)
- Budget display (if available)
- Company name (if provided)

2. Collapsible Job Details

- Collapsed by default
- Click 'View Full Details' to expand
- When expanded shows:
 - * Full job description (scrollable, max 300px height)
 - * Job URL (clickable link, opens new tab)
 - * Budget range
 - * User's proposed price (if set)
 - * Tags

* User notes

AI Settings Panel:

3. Tone Selector

- Label: 'Tone'
- Radio buttons (horizontal layout):
 - Professional (default)
 - Friendly
 - Confident
- Visual: Cards with icons
- Selected state: Blue border + checkmark

4. Length Selector

- Label: 'Proposal Length'
- Radio buttons:
 - Concise (100-150 words)
 - Standard (150-250 words) - default
 - Detailed (250-400 words)
- Helper text: 'Shorter often performs better'

5. Highlights Section

- Label: 'What to highlight'
- Checkboxes (multi-select):
 - Years of experience
 - Similar past projects
 - Fast turnaround time
 - Available immediately
 - Portfolio/case studies
- Up to 3 can be selected
- Disable others after 3 selected

6. Generate Button

- Large, prominent button

- Text: 'Generate Proposal with AI'
- Icon: Magic wand or sparkles
- Background: Blue gradient
- Shows credits remaining:
'X/5 used this month' (free)
'Unlimited' (pro)
- Disabled states:
 - * If no credits left (free plan)
 - * While generating (loading)
- Click handler: calls handleGenerate()

7. Credits Warning (Free Plan Only)

- Shows when credits low:
'Only 2 AI generations left this month'
- Shows when no credits:
'AI limit reached. Upgrade to Pro for unlimited.'
- Upgrade button below

=== RIGHT COLUMN DETAILS ===

Before Generation (Empty State):

- Centered content
- Icon: Document with sparkles
- Heading: 'Ready to generate your proposal'
- Subheading: 'Click generate to create a personalized proposal'
- Tips list:
 - * Be specific about your skills in settings
 - * Shorter proposals often get better responses
 - * You can regenerate if not satisfied

During Generation (Loading State):

- Animated spinner (pulsing blue circle)

- Progress messages (rotate every 2 seconds):

1. 'Analyzing job requirements...'
2. 'Reviewing your profile...'
3. 'Crafting your proposal...'
4. 'Almost done...'

- Typical duration: 4-8 seconds

- Cannot be cancelled once started

After Generation (Success State):

8. Generated Proposal Display

- Large textarea (editable)
- Min height: 400px
- Auto-resize to content
- Font: System default, 14px, line-height 1.6
- Padding: 20px
- Border: Light gray
- Focus state: Blue border

9. Word Count

- Below textarea
- Format: '187 words'
- Color:
 - * Green if within target range
 - * Yellow if too long (>300)
 - * Red if too short (<80)

10. Quality Indicators

- Small panel below word count
- Checkmarks or warnings:
 - ✓ Addresses job requirements
 - ✓ Mentions relevant skills
 - ✓ Appropriate length

✓ Includes call-to-action

■ Consider adding specific examples

- Logic: Simple keyword matching + length check

11. Action Buttons

- Button row (horizontal layout)

A. Copy to Clipboard (Primary)

- Text: 'Copy to Clipboard'

- Icon: Clipboard

- Background: Blue

- onClick: Copy text, show toast

- Success state: Changes to 'Copied!' with checkmark

- Reverts after 3 seconds

B. Regenerate (Secondary)

- Text: 'Regenerate'

- Icon: Refresh

- Background: White, blue border

- onClick: Confirm dialog, then generate again

- Confirmation:

'This will use another AI credit. Continue?'

- Uses new credit

- Keeps same settings

C. Save for Later

- Text: 'Save'

- Icon: Bookmark

- Background: White, gray border

- onClick: Save proposal to DB without using

- Success: Toast 'Proposal saved'

D. Mark as Applied

- Text: 'Mark as Applied'

- Icon: Check circle
- Background: Green
- onClick: Open date picker modal
- Modal shows:
 - * 'When did you apply?'
 - * Date picker (default: today)
 - * Submit button
- On submit:
 - * Update job status to 'applied'
 - * Set applied_at timestamp
 - * Set followup_date (+3 days)
 - * Show success toast
 - * Redirect to jobs list

12. Tips Section

- Collapsible panel
- Header: 'Tips for using this proposal'
- Content:
 - * 'Personalize with specific project details'
 - * 'Add a question to encourage response'
 - * 'Proofread before sending'
 - * 'Paste on platform within 24 hours for best results'

FRONTEND - Component State Management

Component: ProposalGenerator.tsx

State Variables:

```
const [job, setJob] = useState(null)
```

```
// Loaded from API based on job_id param
```

```
const [settings, setSettings] = useState({
```

```
  tone: 'professional',
```

```
  length: 'standard',
```

```
  highlights: []
```

```
})
```

```
const [proposal, setProposal] = useState(null)
```

```
// null = not generated yet
```

```
// string = generated proposal text
```

```
const [editing, setEditing] = useState(false)
```

```
// True if user modified the proposal
```

```
const [generating, setGenerating] = useState(false)
```

```
// True while API call in progress
```

```
const [error, setError] = useState(null)
```

```
// Error message if generation fails
```

```
const [creditsRemaining, setCreditsRemaining] = useState(null)
```

```
// Number of AI credits left (free plan only)
```

```
const [copied, setCopied] = useState(false)
```

```
// True briefly after copy to clipboard
```

Derived State:

```
const canGenerate = useMemo(() => {
```

```
  if (user.plan === 'pro') return true
```

```
  return creditsRemaining > 0
```

```
}, [user.plan, creditsRemaining])
```

```
const wordCount = useMemo(() => {
```

```
if (!proposal) return 0
return proposal.split(/\s+/).length
}, [proposal])

const qualityChecks = useMemo(() => {
  if (!proposal) return []
  // Check various quality criteria
  const checks = []
  if (wordCount >= 100) checks.push('Appropriate length')
  if (proposal.includes('?')) checks.push('Includes question')
  // etc...
  return checks
}, [proposal, wordCount])
```

FRONTEND - Key Functions

1. Load Job Data (useEffect on mount)

```
useEffect(() => {  
  
  const loadJob = async () => {  
  
    try {  
  
      const response = await fetch(`/api/jobs/${job_id}`)  
  
      if (!response.ok) throw new Error('Job not found')  
  
      const data = await response.json()  
  
      setJob(data.job)  
  
  
      // Check if proposal already exists  
  
      if (data.existing_proposal) {  
  
        setProposal(data.existing_proposal.text)  
  
      }  
  
    } catch (error) {  
  
      setError('Failed to load job')  
  
      router.push('/app/jobs')  
  
    }  
  
  }  
  
  loadJob()  
  
}, [job_id])
```

2. Load User Credits

```
useEffect(() => {  
  
  const loadCredits = async () => {  
  
    const { data } = await supabase  
  
      .from('users')  
  
      .select('ai_credits_used, ai_credits_limit, plan')  
  
      .eq('id', user.id)  
  
      .single()  
  
  
    if (data.plan === 'free') {  
  
      setCreditsRemaining(data.ai_credits_limit - data.ai_credits_used)  
  
    }  
  
  }  
  
}
```

```

}

loadCredits()
}, [user.id])

3. Handle Generate

const handleGenerate = async () => {
  // Validate credits
  if (!canGenerate) {
    toast.error('No AI credits remaining. Upgrade to Pro.')
    return
  }

  try {
    setGenerating(true)
    setError(null)

    // Call API
    const response = await fetch('/api/proposals/generate', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        job_id: job.id,
        tone: settings.tone,
        length: settings.length,
        highlights: settings.highlights
      })
    })

    if (!response.ok) {
      const errorData = await response.json()
      throw new Error(errorData.error || 'Generation failed')
    }

    const data = await response.json()

    // Update state

```

```

setProposal(data.proposal)

setCreditsRemaining(data.credits_remaining)

// Success feedback
toast.success('Proposal generated!')

// Track analytics
trackEvent('proposal_generated', {
  job_id: job.id,
  tone: settings.tone,
  length: settings.length
})

} catch (error) {
  setError(error.message)
  toast.error(error.message)
} finally {
  setGenerating(false)
}
}

4. Handle Copy to Clipboard

const handleCopy = async () => {
  try {
    await navigator.clipboard.writeText(proposal)
    setCopied(true)
    toast.success('Copied to clipboard!')

    // Reset after 3 seconds
    setTimeout(() => setCopied(false), 3000)

    // Track
    trackEvent('proposal_copied', { job_id: job.id })

  } catch (error) {
    toast.error('Failed to copy')
  }
}

```

```
}
```

5. Handle Regenerate

```
const handleRegenerate = async () => {
```

```
const confirmed = confirm(
```

```
'This will use another AI credit. Continue?'
```

```
)
```

```
if (!confirmed) return
```

```
// Same as handleGenerate
```

```
await handleGenerate()
```

```
}
```

6. Handle Mark as Applied

```
const handleMarkApplied = async (appliedDate) => {
```

```
try {
```

```
const response = await fetch(`/api/jobs/${job.id}/status`, {
```

```
method: 'PATCH',
```

```
headers: { 'Content-Type': 'application/json' },
```

```
body: JSON.stringify({
```

```
status: 'applied',
```

```
applied_at: appliedDate
```

```
}))
```

```
})
```

```
if (!response.ok) throw new Error('Update failed')
```

```
toast.success('Status updated! Follow-up set for 3 days.')
```

```
router.push('/app/jobs')
```

```
} catch (error) {
```

```
toast.error('Failed to update status')
```

```
}
```

```
}
```

BACKEND SPECIFICATION

API Endpoint: POST /api/proposals/generate

File: app/api/proposals/generate/route.ts

Purpose: Generate AI proposal using OpenAI GPT-4

Request Body:

```
{  
  job_id: string (UUID)  
  tone: 'professional' | 'friendly' | 'confident'  
  length: 'concise' | 'standard' | 'detailed'  
  highlights: string[] (up to 3 items)  
}
```

Response (Success 200):

```
{  
  success: true,  
  proposal: string (generated text)  
  word_count: number  
  credits_remaining: number (for free plan)  
  proposal_id: string (UUID)  
}
```

Response (Error 403 - No Credits):

```
{  
  error: 'AI credit limit reached',  
  message: 'Upgrade to Pro for unlimited generations',  
  upgrade_required: true  
}
```

Response (Error 400):

```
{  
  error: 'Invalid request',  
}
```


details: 'Missing job_id'

}

Response (Error 500):

{

error: 'Generation failed',

message: 'OpenAI API error'

}

BACKEND - Implementation Steps

Step 1: Authenticate User

```
const supabase = createRouteHandlerClient({ cookies })

const { data: { user } } = await supabase.auth.getUser()

if (!user) {
  return NextResponse.json(
    { error: 'Unauthorized' },
    { status: 401 }
  )
}
```

Step 2: Validate Request

```
const body = await request.json()

const { job_id, tone, length, highlights } = body

if (!job_id) {
  return NextResponse.json(
    { error: 'Missing job_id' },
    { status: 400 }
  )
}

if (!['professional', 'friendly', 'confident'].includes(tone)) {
  return NextResponse.json(
    { error: 'Invalid tone' },
    { status: 400 }
  )
}
```

Step 3: Check AI Credits

```
const { data: userProfile } = await supabase
  .from('users')
  .select('plan, ai_credits_used, ai_credits_limit')
  .eq('id', user.id)
```

```

.single()

// Free plan credit check
if (userProfile.plan === 'free') {
  if (userProfile.ai_credits_used >= userProfile.ai_credits_limit) {
    return NextResponse.json(
      {
        error: 'AI credit limit reached',
        message: 'Upgrade to Pro for unlimited generations',
        upgrade_required: true
      },
      { status: 403 }
    )
  }
}

```

Step 4: Fetch Job Details

```

const { data: job, error: jobError } = await supabase
  .from('jobs')
  .select('*')
  .eq('id', job_id)
  .eq('user_id', user.id)
  .single()

if (jobError || !job) {
  return NextResponse.json(
    { error: 'Job not found' },
    { status: 404 }
  )
}

```

Step 5: Fetch User Context

```

const { data: userData } = await supabase
  .from('users')
  .select('skills, experience_level, bio, portfolio_url')

```

```
.eq('id', user.id)
```

```
.single()
```

Step 6: Fetch Similar Past Jobs (for context)

```
// Get past won jobs with similar keywords
```

```
const { data: pastWins } = await supabase
```

```
.from('jobs')
```

```
.select('title, job_description')
```

```
.eq('user_id', user.id)
```

```
.eq('status', 'won')
```

```
.order('created_at', { ascending: false })
```

```
.limit(3)
```

Step 7: Construct System Prompt

```
const systemPrompt = `You are an expert freelance proposal writer.
```

```
Your goal is to write compelling, personalized proposals that win projects.
```

User Context:

```
- Skills: ${userData.skills.join(', ')}
```

```
- Experience Level: ${userData.experience_level}
```

```
- Bio: ${userData.bio || 'Not provided'}
```

```
- Portfolio: ${userData.portfolio_url || 'Not provided'}
```

Past Successes:

```
${pastWins.map(w => `- Won: ${w.title}`).join('\n')}
```

Writing Guidelines:

```
1. Tone: ${tone}
```

```
2. Length: ${getLengthGuidance(length)}
```

```
3. Highlight: ${highlights.join(', ')}
```

```
4. Be specific and relevant to job requirements
```

```
5. Show enthusiasm but remain professional
```

```
6. Include a soft call-to-action
```

```
7. Do NOT include:
```

```
- Greeting (user will add manually)
```

```
- Pricing discussion (unless job asks)
```

- Generic platitudes

8. DO include:

- Direct response to job requirements
- 2-3 relevant skills or experiences
- One question to encourage response

Format: Plain text, no markdown, no bold/italics`

Step 8: Construct User Prompt

```
const userPrompt = `Job Title: ${job.title}
```

```
Platform: ${job.platform}
```

```
Budget: ${job.budget_min}-${job.budget_max} ${job.currency}
```

```
Job Description:
```

```
${job.job_description}
```

```
Write a proposal for this job now.`
```

Step 9: Call OpenAI API

```
const openai = new OpenAI({
```

```
  apiKey: process.env.OPENAI_API_KEY
```

```
})
```

```
const completion = await openai.chat.completions.create({
```

```
  model: 'gpt-4',
```

```
  messages: [
```

```
    { role: 'system', content: systemPrompt },
```

```
    { role: 'user', content: userPrompt }]
```

```
  ],
```

```
  temperature: 0.7,
```

```
  max_tokens: getMaxTokens(length),
```

```
  presence_penalty: 0.6,
```

```
  frequency_penalty: 0.3
```

```
})
```

```
const generatedText = completion.choices[0].message.content
```

Step 10: Save Proposal to Database

```

const { data: savedProposal } = await supabase
  .from('proposals')
  .insert({
    job_id: job_id,
    user_id: user.id,
    generated_text: generatedText,
    tone: tone,
    length: length,
    highlights: highlights,
    tokens_used: completion.usage.total_tokens,
    model_used: 'gpt-4',
    version: 1
  })
  .select()
  .single()

```

Step 11: Update User Credits

```

if (userProfile.plan === 'free') {
  await supabase
    .from('users')
    .update({
      ai_credits_used: userProfile.ai_credits_used + 1
    })
    .eq('id', user.id)
}

```

Step 12: Track Analytics

```

await supabase.from('events').insert({
  user_id: user.id,
  event_type: 'proposal_generated',
  metadata: {
    job_id,
    tone,
    length,

```

```
word_count: generatedText.split(/\s+/).length
}
})
```

Step 13: Return Response

```
return NextResponse.json({
  success: true,
  proposal: generatedText,
  word_count: generatedText.split(/\s+/).length,
  credits_remaining: userProfile.plan === 'pro'
    ? null
    : (userProfile.ai_credits_limit - userProfile.ai_credits_used - 1),
  proposal_id: savedProposal.id
})
```

BACKEND - Helper Functions

```
function getLengthGuidance(length: string): string {  
  switch (length) {  
    case 'concise':  
      return '100-150 words. Be brief and punchy.'  
    case 'standard':  
      return '150-250 words. Balanced detail.'  
    case 'detailed':  
      return '250-400 words. Comprehensive but not verbose.'  
    default:  
      return '150-250 words'  
  }  
}
```

```
function getMaxTokens(length: string): number {  
  switch (length) {  
    case 'concise': return 250  
    case 'standard': return 400  
    case 'detailed': return 600  
    default: return 400  
  }  
}
```

```
async function handleOpenAIError(error: any) {  
  if (error.status === 429) {  
    return {  
      error: 'Rate limit exceeded',  
      message: 'Too many requests. Please try again in a moment.'  
    }  
  }  
}
```

```
if (error.status === 401) {  
  return {  
    error: 'API configuration error',  
    message: 'Please contact support'  
  }  
}
```



```
}  
}  
  
return {  
  error: 'Generation failed',  
  message: 'An error occurred. Please try again.'  
}  
}
```

DATABASE SPECIFICATION

Table: proposals

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Unique proposal ID
job_id	UUID	FK jobs(id)	Associated job
user_id	UUID	FK users(id)	Proposal owner
generated_text	TEXT	NOT NULL	AI-generated content
tone	VARCHAR(20)		professional/friendly/confident
length	VARCHAR(20)		concise/standard/detailed
highlights	TEXT[]	DEFAULT '{}'	Selected highlights
version	INTEGER	DEFAULT 1	If regenerated
tokens_used	INTEGER		OpenAI tokens consumed
model_used	VARCHAR(50)	DEFAULT 'gpt-4'	AI model
was_edited	BOOLEAN	DEFAULT false	User modified
created_at	TIMESTAMPTZ	DEFAULT NOW()	Generation time

SQL Schema:

```
CREATE TABLE proposals (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  job_id UUID NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  generated_text TEXT NOT NULL,  
  tone VARCHAR(20),  
  length VARCHAR(20),  
  highlights TEXT[] DEFAULT '{}',  
  version INTEGER DEFAULT 1,  
  tokens_used INTEGER,  
  model_used VARCHAR(50) DEFAULT 'gpt-4',  
  was_edited BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Indexes:

```
CREATE INDEX idx_proposals_job_id ON proposals(job_id);  
CREATE INDEX idx_proposals_user_id ON proposals(user_id);  
CREATE INDEX idx_proposals_created_at ON proposals(created_at DESC);
```

RLS Policies:

```
ALTER TABLE proposals ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY 'Users view own proposals'  
ON proposals FOR SELECT  
USING (auth.uid() = user_id);
```

```
CREATE POLICY 'Users insert own proposals'  
ON proposals FOR INSERT  
WITH CHECK (auth.uid() = user_id);
```

```
CREATE POLICY 'Users update own proposals'  
ON proposals FOR UPDATE  
USING (auth.uid() = user_id);
```

Queries:

```
-- Get proposal for job  
SELECT * FROM proposals  
WHERE job_id = $1 AND user_id = $2  
ORDER BY version DESC  
LIMIT 1;  
  
-- Count proposals this month  
SELECT COUNT(*) FROM proposals  
WHERE user_id = $1  
AND created_at >= date_trunc('month', CURRENT_DATE);  
  
-- Get all proposals with job info  
SELECT  
p.*,  
j.title as job_title,
```

```
j.platform,  
j.status as job_status  
FROM proposals p  
JOIN jobs j ON j.id = p.job_id  
WHERE p.user_id = $1  
ORDER BY p.created_at DESC;
```